RegNo: 223026689 **date:29th/03/2026**

UR CST

COMPUTER ENGINEERING

YEAR 3

SEMESTER II

GAKO CAMPUS

Lab 4: Consuming APIs in flutter:

This lab contain three sections :

- Scann of handwritten answers
- Screenshots of app
- Code for app (uploded on github)

Reg N° : 223026689

Flutter Lab 4 : Consuming APIs in Flutter

Question 1. Which dependencies did you use and why?

Dependencies	Purpose
Http	Used to make HTTP requests (GET , POST , PUT , DELETE) to the JSONPlaceholder API . This is the most commonly used package for API calls in flutter
Provider	Used for state management . It helps manage the app state efficiently and notifies widgets when data changes (e.g: when post are fetched , created or deleted).
intl	Used for date formatting if needed (though not heavily used in this implementation, it's included for potential future enhancement like formating time stamps .

Why these ?

http : Simple, well maintained and provides all necessary methods for REST API integration .

Provider : Light weight, follows flutter's reactive pattern and avoids boilerplate code compared to other state management solutions.

Intl : Useful for internationalization and formatting dates if the API includes timestamp fields.

Question 2: Generally, how can you handle API exceptions (network errors, invalid data)?

Exceptional handling strategy

1. Try-Catch blocks: Wrap all API calls in try-catch block to catch exceptions like SocketException (no internet), timeoutException or HttpException.

2. HTTP status code validation: Check response status code:

   200/201 → Success
   400 → Bad request (invalid data)
   404 → Not found
   500 → Server error.

3. Error states in UI

   - Show loading indicators during requests
   - Display error messages with retry buttons
   - Use SnackBar for non critical errors.

4. User feedback: Show friendly error messages and provide retry options.

Scanned with CamScanner

Question 3: Explain the Future widget have you used. And a part of the UI depends on the future.

Future in Flutter

A future represents a potential value or error that will be available at some time in the future. it's used for asynchronous operation like API call.

Where Future is used in this app :

1. FutureBuilder (alternative approach - though not directly used in this implementation) ~~It's used~~

code

```
FutureBuilder <List<Post>> (
    future: apiservice.fetchPosts(),
    builder: (context, snapshot) {
    if (snapshot.connectionState = connectionState.waiting) {
        return circularProgressIndicator(); // Loading state
    } else if (snapshot.hasError) {
        return ErrorWidget (snapshot.error); // Error state
    } elseif (snapshot.hasdata) {
        return ListView (...) // data loaded.
    }
    return Container();
    },
)
```

2. UI parts that depend on Future results :
   - Loading Indicator : shown while waiting for API response
   - Post list : displayed after data is successfully fetched
   - Error message : shown if Future completes with error
   - Success messages : show after create/update/delete operations

Scanned with CamScanner

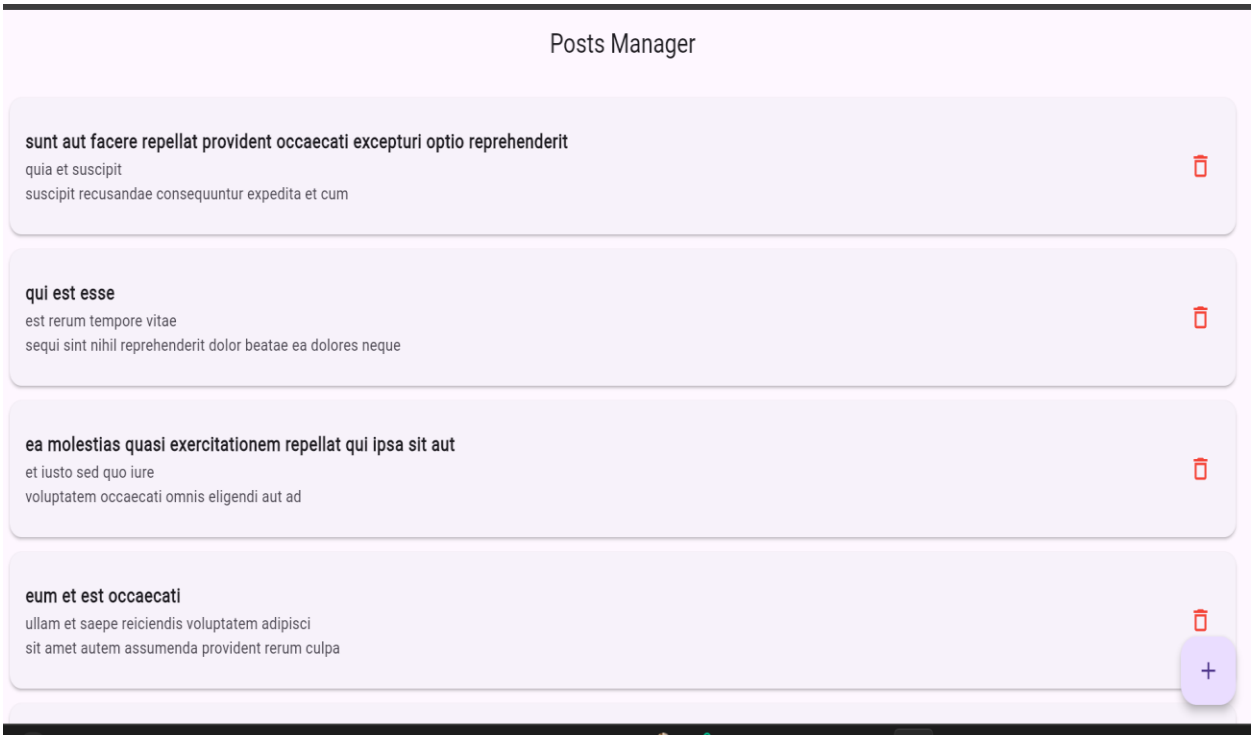
State Management with provider:

Instead of using FutureBuilder directly in widgets, this app us
provider with async methods. the UI reacts to:

- _isLoading: shows loading spinner
- _posts: displays the list
- _errorMessage: shows error widget.

Screenshot 1

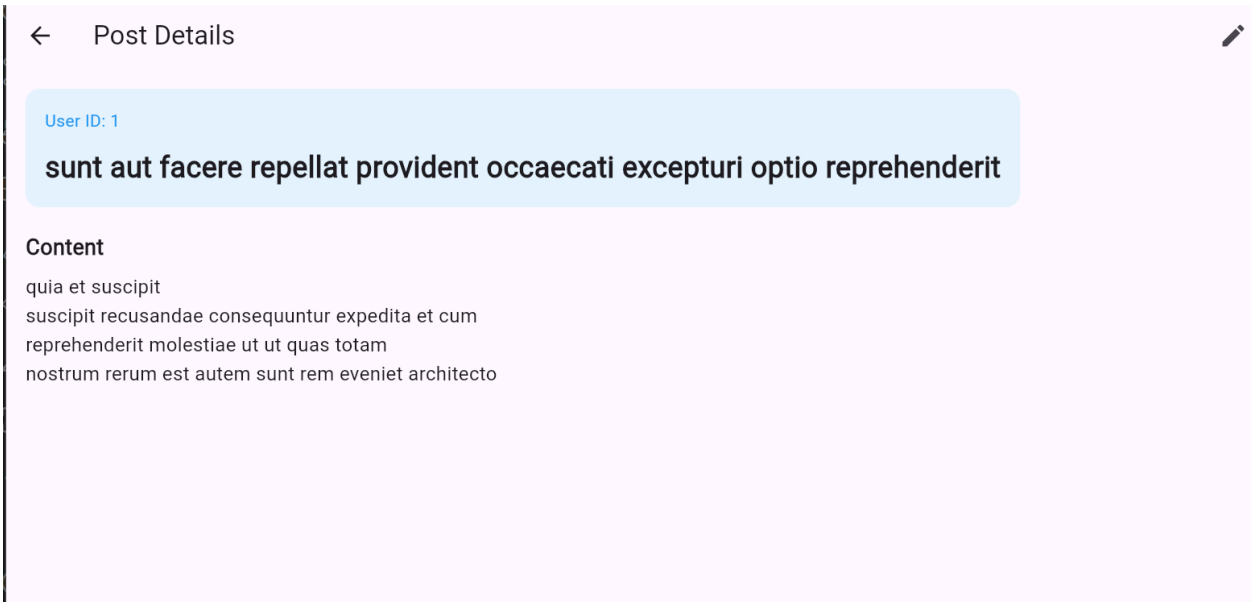
This screenshot shows the main screen of the Posts Manager app displaying a list of posts fetched from the JSONPlaceholder API.



(screenshot.1)

Screenshot 2

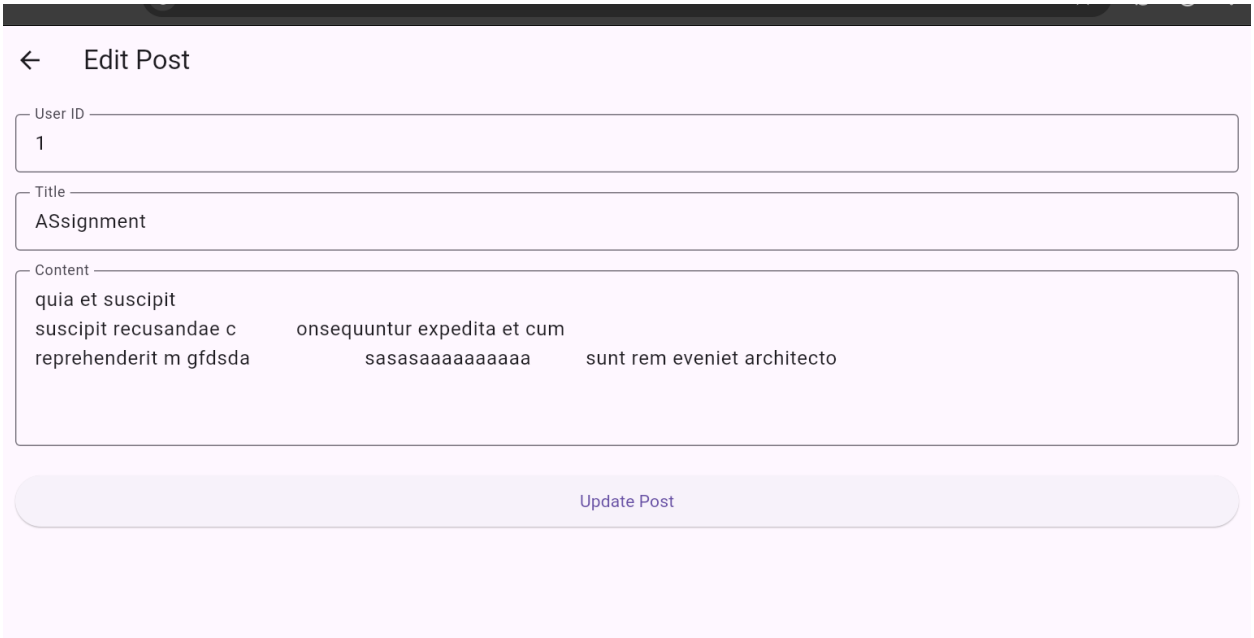
This screenshot displays the detailed view of a single post, showing complete information when a user taps on a post from the list.



(screenshot.2)

Screenshot 3

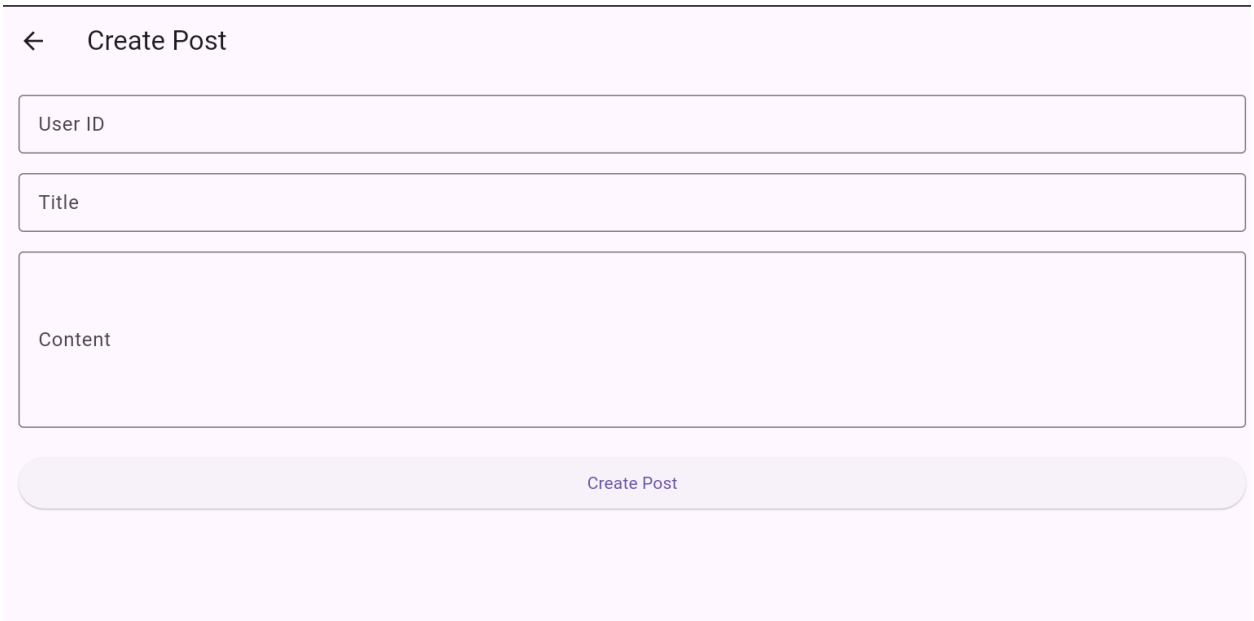
This screenshot shows the edit form with pre-filled data from an existing post, allowing users to modify and update the post.



(screenshot.3)

Screenshot 4

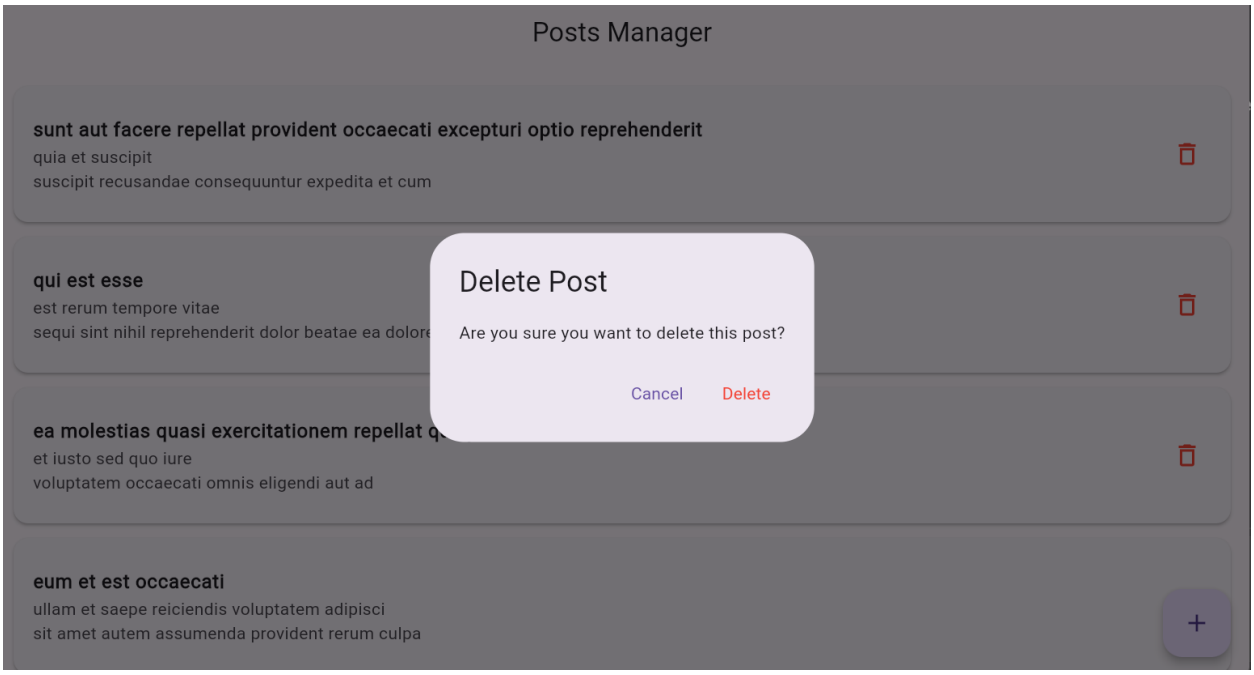
This screenshot shows the form interface for creating a new post, with input fields for user ID, title, and content.



(screenshot.4)

SCREENSHOT 5

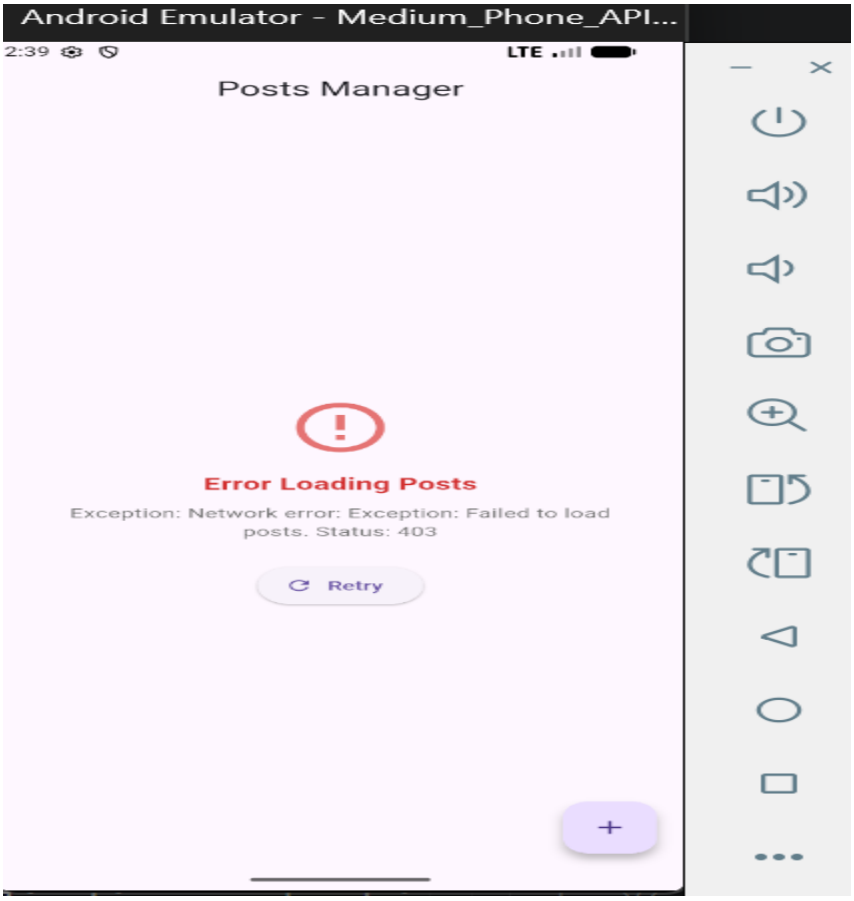
This screenshot shows a confirmation dialog that appears when a user attempts to delete a post, asking for confirmation before permanent deletion.



(screenshot.5)

SCREENSHOT 6

This screenshot shows the error screen displayed when network issues occur, such as no internet connection or API server errors.



(screenshot.6)